

Introducción a Whoosh

Modelo booleano clásico y motor de búsqueda en Python

Erik Figueiral Alonso · erik.figueiral.alonso@uvigo.gal

Práctica guiada en clase

NO SE ENTREGA

Índice

- | | | |
|----------|-----------------------|---------------------------------|
| 1 | Introducción | ¿Qué es Whoosh? |
| 2 | Arquitectura | Pipeline y módulos de la API |
| 3 | Representación | Schema y análisis textual |
| 4 | Indexación | Writer y Storage |
| 5 | Búsqueda | QueryParser, Searcher y Results |
| 6 | Práctica | Recorrido del notebook |

01

¿Qué es Whoosh?

Un motor de búsqueda escrito en Python para crear índices y consultar colecciones documentales.

En este bloque conectaremos la API con el pipeline completo de Recuperación de Información.

De programarlo nosotros a delegarlo en el motor

ANTES · IMPLEMENTACIÓN MANUAL

Programamos la tokenización

Construimos término → postings

Programamos AND, OR y resultados

DELEGAMOS

DELEGAMOS

DELEGAMOS

AHORA · WHOOSH LO GESTIONA

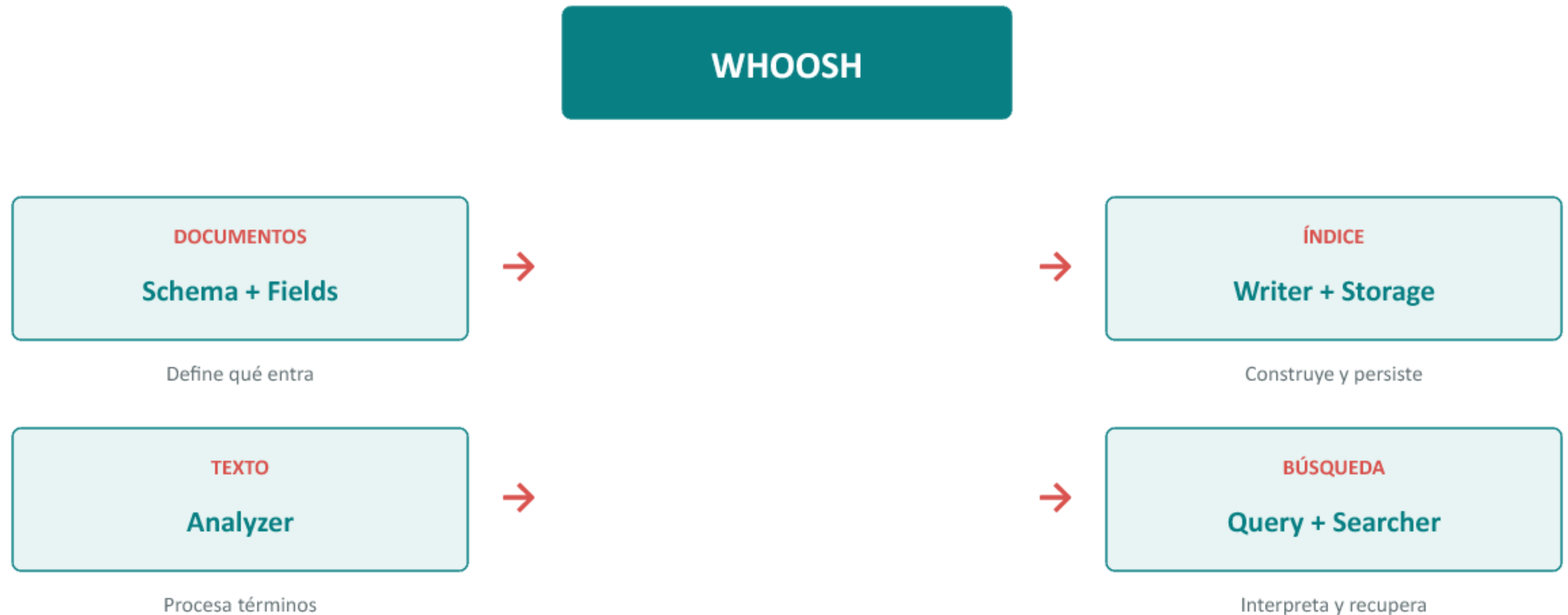
Configuramos el Analyzer

Definimos Schema, Writer y Storage

Expresamos Query y usamos Searcher

Las decisiones siguen siendo nuestras; Whoosh ejecuta y mantiene la infraestructura.

Whoosh organiza el sistema en objetos conectados



Configuramos estos objetos; el motor coordina el flujo completo.

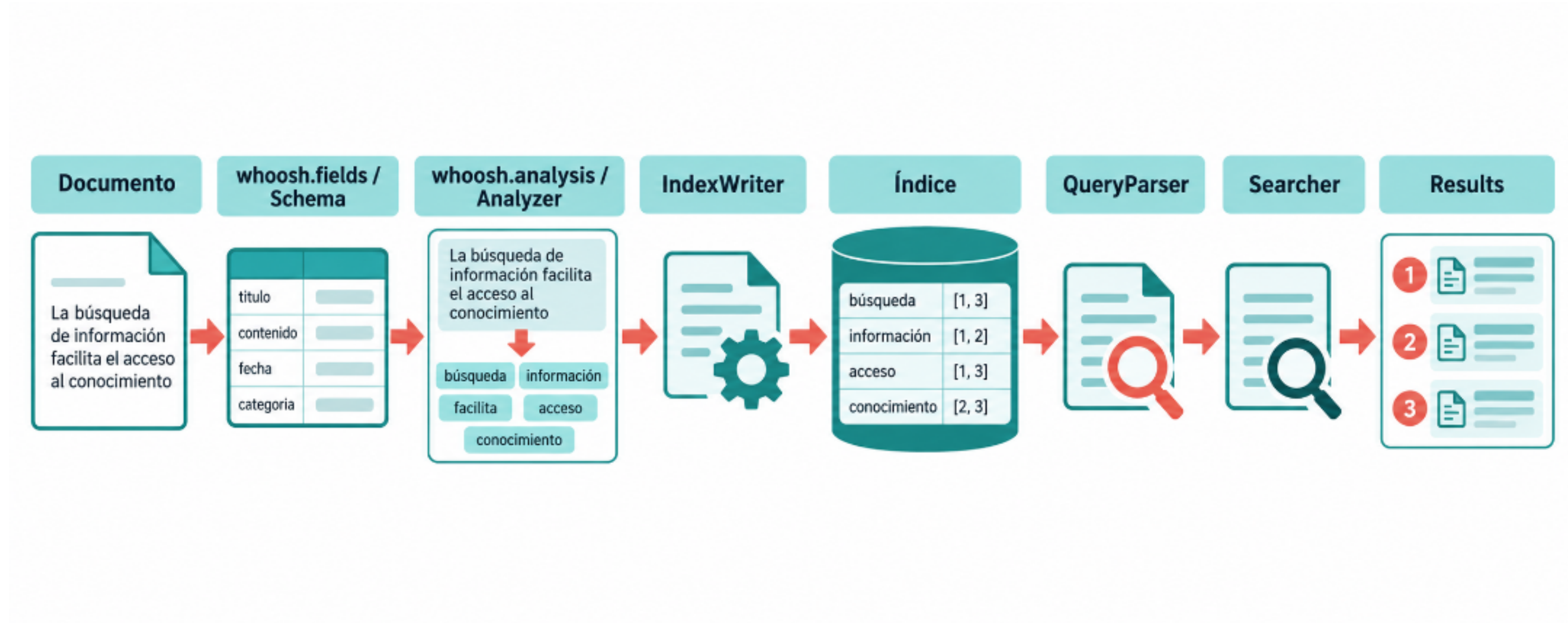
02

Arquitectura y pipeline

Dos recorridos conectados: indexación fuera de línea y consulta en línea.

En este bloque conectaremos la API con el pipeline completo de Recuperación de Información.

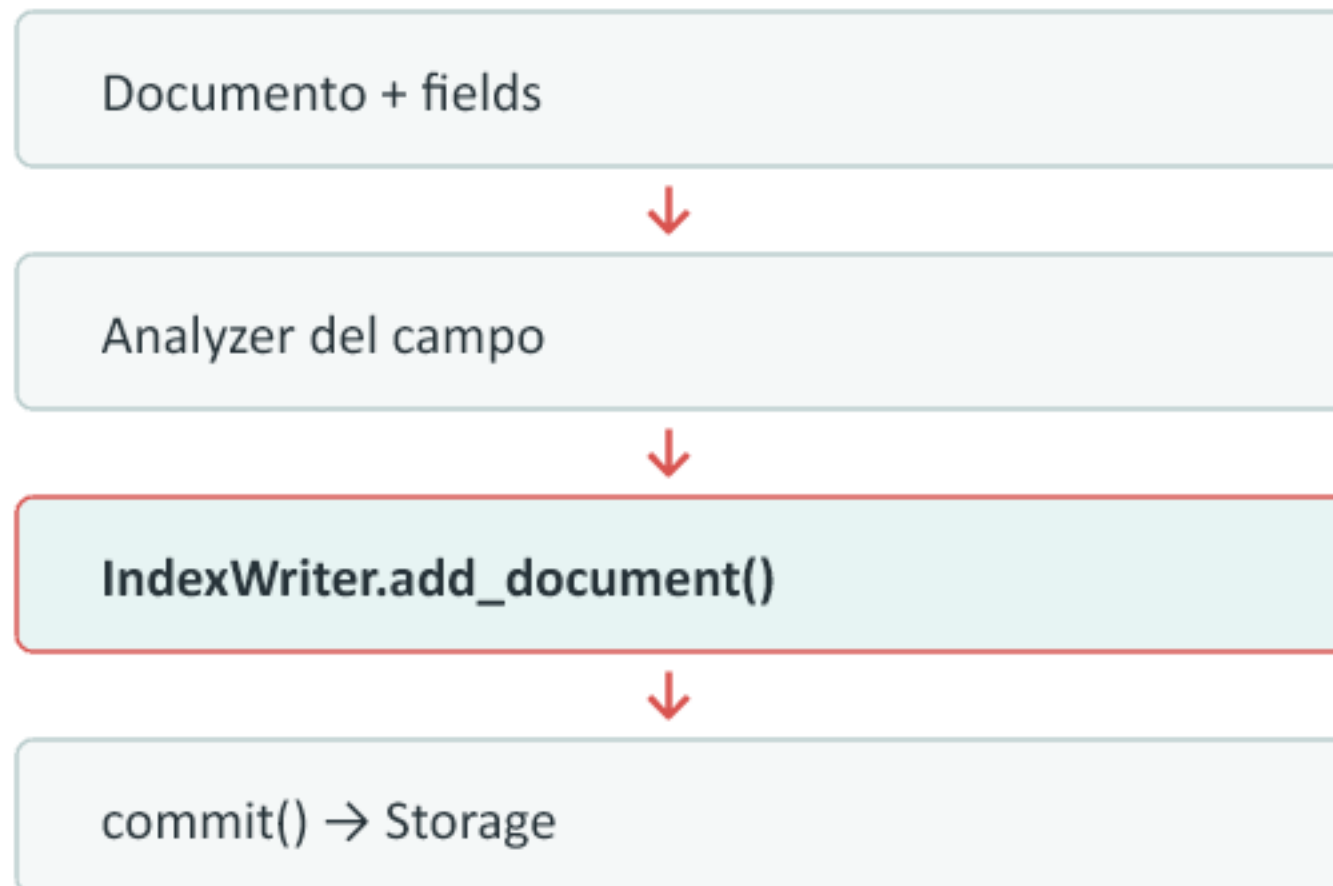
La arquitectura completa de Whoosh



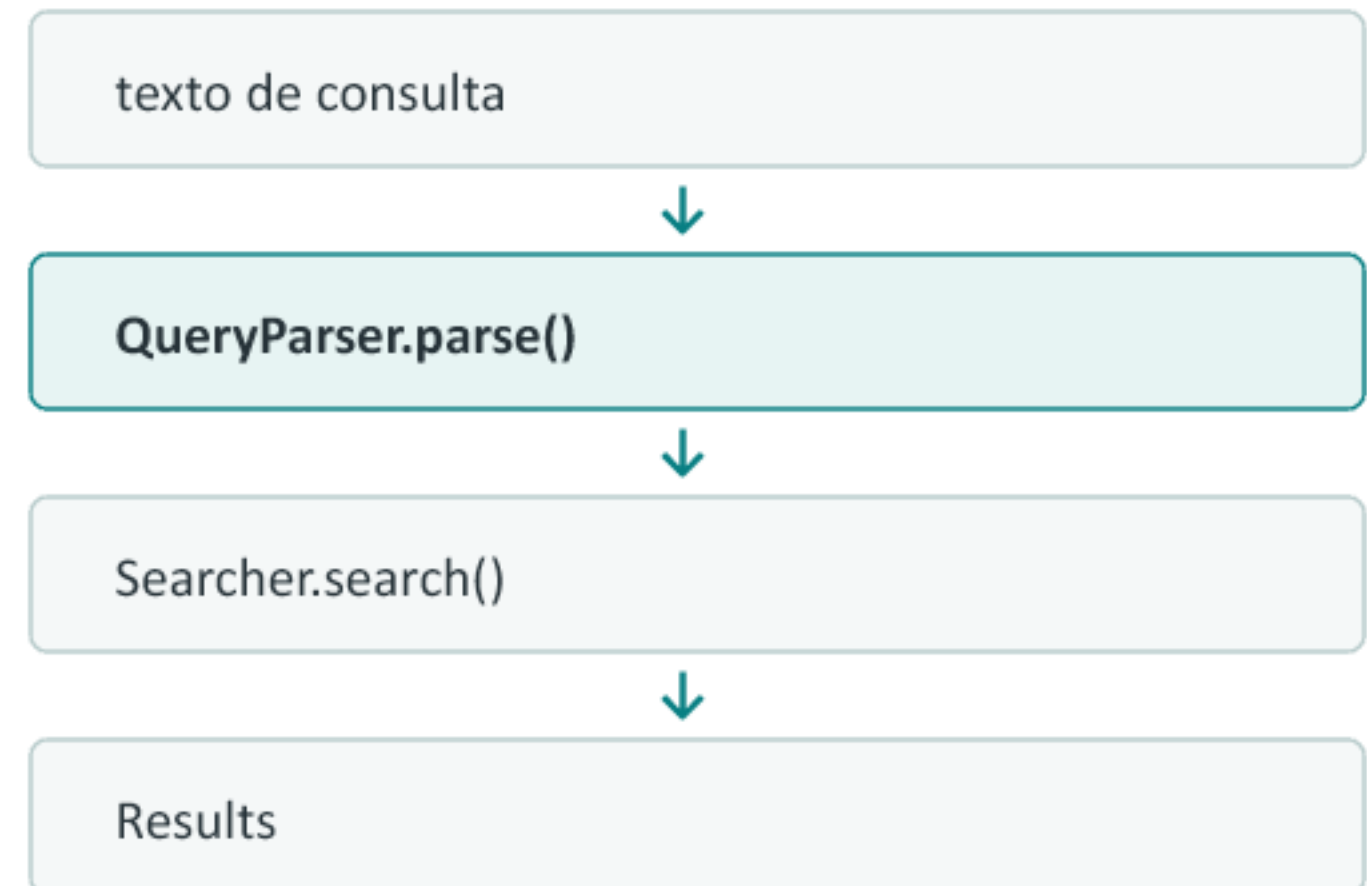
Índice y consulta deben compartir la misma política de análisis.

Dos procesos que comparten el índice

INDEXACIÓN · fuera de línea



BÚSQUEDA · en línea



La API de Whoosh: seis módulos, un flujo

whoosh.fields

Schema · TEXT · ID

¿Qué campos tiene el documento?

```
Schema(title=TEXT(stored=True))
```

whoosh.analysis

Analyzer · filtros

¿Cómo se transforma el texto?

```
RegexTokenizer() | LowercaseFilter()
```

whoosh.index

Index

¿Dónde está el índice?

```
create_in() · open_dir()
```

whoosh.writing

Writer

¿Cómo se añaden documentos?

```
add_document() · commit()
```

whoosh.qparser

QueryParser

¿Cómo se interpreta la consulta?

```
parse("python AND datos")
```

whoosh.searching

Searcher · Results

¿Cómo se recuperan los hits?

```
search(query) · hit["title"]
```

Cada módulo responde a una decisión concreta del sistema de RI.

03

Schema y análisis textual

Diseñamos la representación del documento antes de indexarlo.

En este bloque conectaremos la API con el pipeline completo de Recuperación de Información.

Schema: cada campo declara su función

```
from whoosh.fields import Schema, ID, TEXT, KEYWORD
```

```
from whoosh.analysis import LanguageAnalyzer
```

```
analizador_es = LanguageAnalyzer("es")
```

```
schema = Schema(
```

```
    codigo=ID(stored=True, unique=True),
```

```
    titulo=TEXT(stored=True),
```

```
    contenido=TEXT(analyzer=analizador_es),
```

```
    temas=KEYWORD(commas=True, stored=True)
```

```
)
```

EJEMPLO INDEXADO

```
writer.add_document(
```

```
    codigo="001",
```

```
    titulo="Recuperación documental",
```

```
    contenido="Los sistemas recuperaron documentos",
```

```
    temas="índices, búsqueda"
```

```
)
```

contenido no se almacena

pero sí se analiza e indexa

recuperaron → recuper

`LanguageAnalyzer("es")` aplica minúsculas, stopwords y stemming del español.

Tipos de field: la pregunta que responde cada uno

TEXT

¿Qué palabras contiene?

```
contenido="Python procesa documentos"
```

tokeniza · posiciones · frases

ID

¿Cuál es su identidad?

```
codigo="DOC-001"
```

valor completo · unique

KEYWORD

¿Qué etiquetas tiene?

```
temas="python, recuperación"
```

separa por espacios o comas

STORED

¿Qué quiero mostrar?

```
resumen={...}
```

se guarda · no se busca

NUMERIC

¿Qué valor numérico tiene?

```
precio=19.99
```

rango · ordenación

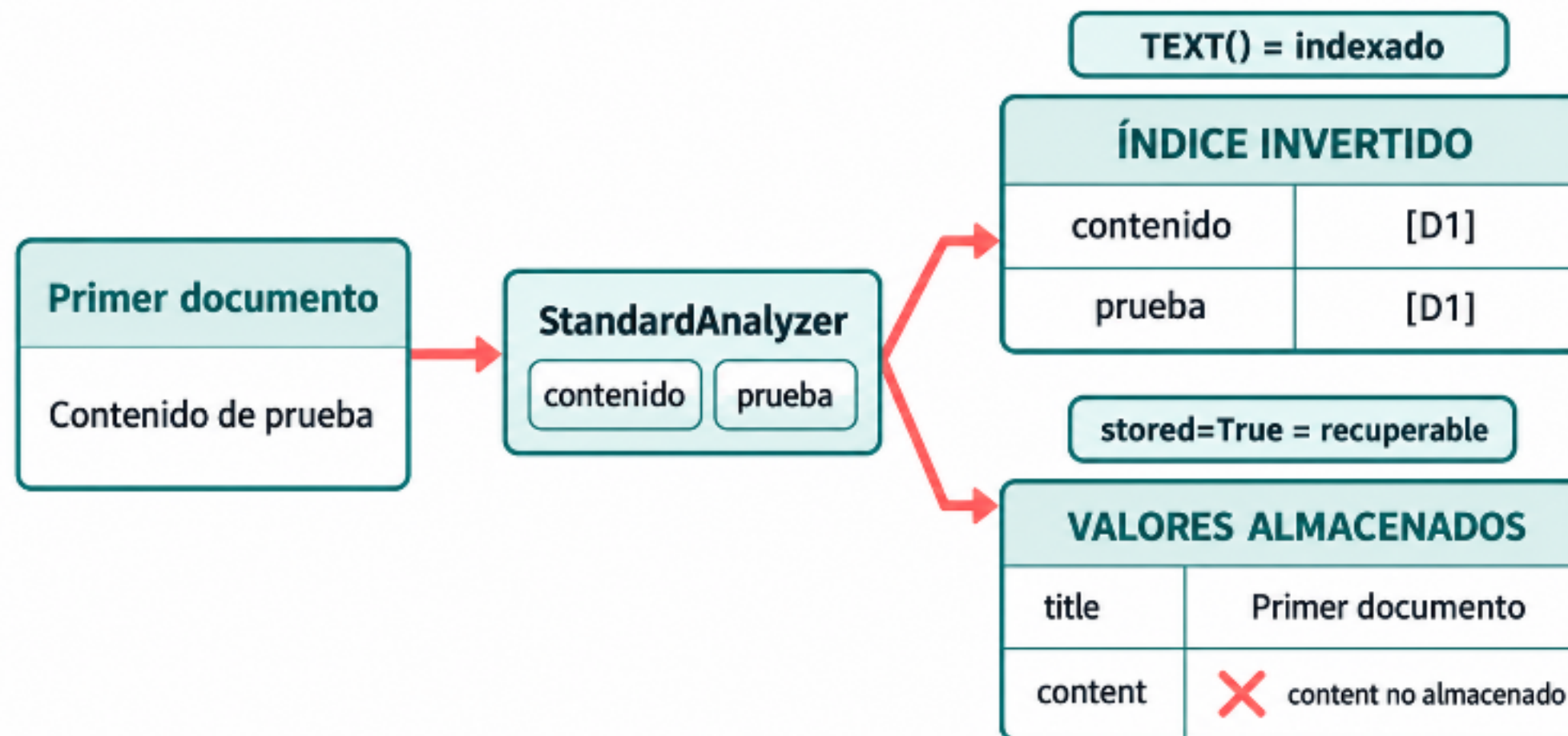
DATETIME

¿Cuándo ocurrió?

```
fecha=datetime(2026, 10, 9)
```

intervalos temporales

TEXT(): indexar no significa almacenar



TEXT() usa StandardAnalyzer por defecto. stored=True solo decide si recuperamos el valor original.

Cada campo TEXT puede tener su propio Analyzer

```
from whoosh.analysis import StandardAnalyzer, LanguageAnalyzer
```

```
schema = Schema(  
    titulo=TEXT(  
        stored=True,  
        analyzer=StandardAnalyzer(stoplist=None)  
    ),  
    contenido=TEXT(  
        analyzer=LanguageAnalyzer("es")  
    )  
)
```

DOS CAMPOS · DOS POLÍTICAS

título conserva palabras funcionales

"Modelos de recuperación"

→ modelos · de · recuperación

contenido stemming español

Índice: "recuperaron documentos"

Consulta: "recuperar documento"

ambos → recuper · document

El mismo Analyzer procesa documentos y consultas: las variantes morfológicas pueden coincidir.

04

Indexación y almacenamiento

El writer transforma documentos y el storage conserva el índice.

En este bloque conectaremos la API con el pipeline completo de Recuperación de Información.

Crear y abrir un índice persistente

```
import os
from whoosh import index

os.makedirs("indexdir", exist_ok=True)

if index.exists_in("indexdir"):
    ix = index.open_dir("indexdir")
else:
    ix = index.create_in("indexdir", schema)
```

- `create_in()` crea el índice y guarda el schema.
- `open_dir()` abre un índice existente.
- `exists_in()` evita recrearlo por error.
- `FileStorage` usa archivos en una carpeta.

Crear de nuevo ≠ abrir lo existente

IndexWriter: una transacción de escritura

```
writer = ix.writer()

writer.add_document(
    id="D1",
    title="Índices invertidos",
    body="Las listas de postings..."
)

writer.commit()
```



writer()

abre el escritor y bloquea la escritura



add_document()

valida los fields y analiza TEXT



commit()

guarda los cambios y libera el writer

Sin commit(), los cambios no quedan confirmados.

Storage: dónde vive el índice

FileStorage

```
from whoosh.filedb.filestore import FileStorage

storage = FileStorage("indexdir")
ix = storage.create_index(schema)
```

Persistente · archivos en disco
Adecuado para la práctica y para reabrir el índice.

RamStorage

```
from whoosh.filedb.filestore import RamStorage

storage = RamStorage()
ix = storage.create_index(schema)
```

Temporal · completamente en memoria
Rápido para pruebas; desaparece al terminar el proceso.

BufferedWriter: commit() no significa terminar

IndexWriter

```
writer = ix.writer()  
writer.add_document(...)  
writer.commit()  
  
# cambios confirmados y writer cerrado
```

flujo corto

abrir → añadir → commit → fin

BufferedWriter

```
writer = BufferedWriter(ix, limit=2)  
writer.add_document(...)  
writer.commit() # puede seguir abierto  
writer.close() # finaliza y libera lock
```

flujo continuo

añadir → commit → añadir → close()

Si vuelves a crear otro BufferedWriter mientras el anterior conserva el bloqueo, puede aparecer LockError. Al terminar: close().

05

Consultas y resultados

Convertimos una necesidad de información en un objeto Query ejecutable.

En este bloque conectaremos la API con el pipeline completo de Recuperación de Información.

QueryParser: de texto a un objeto Query

```
from whoosh.qparser import QueryParser

parser = QueryParser("body", schema=ix.schema)
query = parser.parse(
    '(recuperación OR postings) AND documentos'
)

print(query)
```

- El primer argumento es el campo por defecto.
- El schema permite analizar cada término con la política del campo.
- AND, OR, NOT y paréntesis expresan la lógica booleana.
- También admite campos explícitos: title:índice.

La consulta textual no se ejecuta directamente: primero se interpreta.

Consultas: el lenguaje y los objetos representan lo mismo

Cadena para el usuario

"recuperación AND documentos"

Objeto para el motor

```
And([
  Term("body", "recuperación"),
  Term("body", "documentos")
])
```

OR

unión de postings

AND

intersección

NOT

exclusión

"frase exacta"

posiciones

recuper*

prefijo

date:[2020 TO 2024]

rango

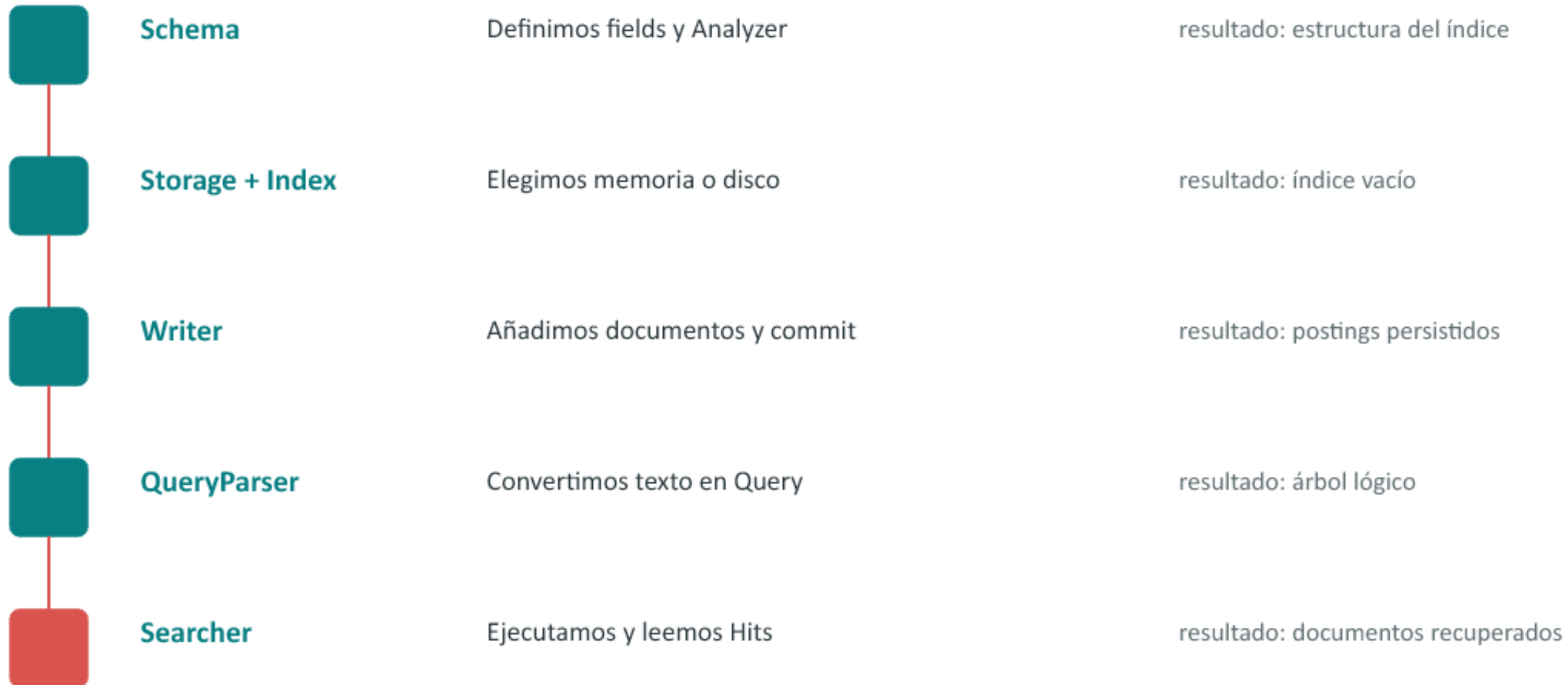
Searcher y Results: ejecutar, limitar y mostrar

```
with ix.searcher() as searcher:  
    results = searcher.search(query, limit=10)  
  
    for hit in results:  
        print(hit["title"], hit.score)
```

- with cierra el Searcher automáticamente.
- search() recibe un Query, no una cadena.
- Results se comporta como una colección de hits.
- Solo se puede mostrar un field almacenado.
- limit=None recupera todo, pero puede costar más.

INDEXADO responde «¿coincide?» · ALMACENADO responde «¿qué puedo mostrar?»

Recorrido del notebook: qué construimos en cada paso



El notebook recorre el ciclo completo: definir → indexar → consultar → recuperar.

Depuración: síntoma → causa → corrección

SÍNTOMA	CAUSA	CORRECCIÓN
No aparece hit["contenido"]	TEXT no tiene stored=True	Almacena el campo que quieras mostrar.
La consulta devuelve 0 resultados	Analyzer distinto o campo incorrecto	Inspecciona tokens y usa el schema del índice.
Al reejecutar cambia el comportamiento	Se recrea o bloquea el índice	exists_in/open_dir y un solo writer.
No se guardan documentos	Falta commit()	Confirma o cancela la transacción.
Falla DateRange / NumericRange	Tipo de field o valor incorrecto	Usa DATETIME/NUMERIC y objetos compatibles.

Diagnostica el flujo: campo → análisis → escritura → consulta → lectura.

Ejemplo completo · 1/2 · documento → índice

```
from whoosh.fields import Schema, ID, TEXT
```

```
from whoosh.analysis import LanguageAnalyzer
```

```
from whoosh.filedb.filestore import RamStorage
```

1. El contenido usa stemming en español

```
schema = Schema(  
    codigo=ID(stored=True),  
    titulo=TEXT(stored=True),  
    contenido=TEXT(analyzer=LanguageAnalyzer("es"))  
)
```

2. Creamos el índice en memoria

```
storage = RamStorage()  
ix = storage.create_index(schema)  
writer = ix.writer()
```

3. Añadimos variantes morfológicas

```
writer.add_document(  
    codigo="001",  
    titulo="Sistemas documentales",  
    contenido="El sistema recuperó documentos relevantes"  
)
```

```
writer.add_document(  
    codigo="002",  
    titulo="Construcción de índices",  
    contenido="Los índices permiten buscar información"  
)
```

Confirma cambios y libera el writer

```
writer.commit()
```

Ejemplo completo · 2/2 · consulta booleana

```
from whoosh.qparser import QueryParser
```

4. El parser aplica el Analyzer de contenido

```
parser = QueryParser("contenido", schema=ix.schema)
consulta = parser.parse(
    "recuperar AND documentos"
)
```

```
print(consulta)
```

```
# (contenido:recuper AND contenido:document)
```

5. Ejecutamos la consulta

```
with ix.searcher() as searcher:
    resultados = searcher.search(consulta)
    for hit in resultados:
        print(hit["codigo"], hit["titulo"])
```

RESULTADO

001 Sistemas documentales

1

recuperar

→ recuper

2

documentos

→ document

3

AND

exige ambos términos

4

Hit

devuelve campos
stored

Idea final: Whoosh hace operativo el modelo de RI

DOCUMENTOS



REPRESENTACIÓN



ÍNDICE



RESULTADOS

**Schema decide qué existe.
Analyzer decide qué términos representan el texto.
Writer construye el índice.
QueryParser y Searcher lo convierten en búsqueda.**

La práctica consiste en conectar estas decisiones, no solo en ejecutar celdas.

Referencias para la práctica

- Whoosh 2.7.4 · Quick start — whoosh.readthedocs.io/en/latest/quickstart.html
- Designing a schema — whoosh.readthedocs.io/en/latest/schema.html
- How to index documents — whoosh.readthedocs.io/en/latest/indexing.html
- How to search — whoosh.readthedocs.io/en/latest/searching.html

Documentación oficial consultada el 9 de octubre de 2026.